

Relatorio Baseline do Pipeline de Auditoria

Smart Contract Security Audit Pipeline - estado antes da reorganizacao

Campo	Valor
Data de geracao	23 May 2026
Branch ativa	develop
Commit HEAD	fda0435
main	fda0435
develop	fda0435
Backup local	stash@{Sat May 23 10:12:01 2026}: On main: baseline-before-develop-align-2026-05-23

Este documento registra o estado operacional do pipeline antes das proximas refatoracoes. Ele foi gerado apos alinhar a branch develop com a main, mantendo a main como referencia confiavel e preservando as mudancas locais em stash.

1. Objetivo do Pipeline

O projeto automatiza parte do fluxo de auditoria e correcao de contratos Solidity. A ideia central e combinar analise estatica, especificacoes formais, verificacao matematica e uma LLM para diagnosticar falhas e propor correcoes minimas.

- Detectar vulnerabilidades em contratos Solidity usando Slither.
- Normalizar achados em um formato estruturado com IDs VULN_XXX.
- Associar vulnerabilidades a propriedades formais e padroes CVL quando aplicavel.
- Gerar ou preparar uma spec CVL para o Certora Prover.
- Verificar o contrato original, diagnosticar violações e gerar uma versao corrigida.
- Revalidar o contrato corrigido em loop, ate 3 tentativas.

2. Arquitetura Atual

Componente	Arquivo principal	Responsabilidade
Orquestrador	agent/orchestrator.py	Controla o fluxo Slither -> LLM -> CVL -> Certora -> diagnostico -> fix -> revalidacao.
Cliente LLM	agent/llm/client.py	Usa OpenAI SDK apontando para a API Groq, com modelo llama-3.3-70b-versatile.
Schemas	agent/llm/schemas.py	Define contratos Pydantic para saidas estruturadas da LLM.
Slither wrapper	agent/tools/slither_cli.py	Executa Slither, filtra achados e adiciona mapeamentos/fallbacks.
Certora wrapper	agent/tools/certora_cli.py	Executa certoraRun contra contrato e spec CVL.
Spec validator	agent/tools/spec_validator.py	Remove rules vazias, deduplica e aplica autocorrecoes sintaticas em CVL.
Prompts	agent/prompts/system_prompts.py	Centraliza instrucoes de normalizacao, geracao de spec, analise, diagnostico e correcao.

3. Etapas Operacionais

Etapa 1 - Slither: `run_slither` executa `slither --json --no-fail-pedantic`. O resultado bruto é minificado e enriquecido com tipo, impacto, confiança, elementos e possível propriedade formal.

Etapa 1b - Normalização LLM: A LLM recebe o JSON minificado e o contrato fonte, e deve retornar `RelatorioSlitherNormalizado` via `Pydantic`.

Etapa 3 - Geração de Spec CVL: A LLM gera `CodigoGerado` contendo CVL. O pipeline aplica `remover_rules_vazias`, `deduplicar_spec`, `corrigir_spec` e `validar_spec`.

Certora Original: `run_certora` executa `certoraRun` no contrato original e salva `certora_original_log.txt`.

Etapa 4 - Análise do Log: A LLM cruza log do Certora com vulnerabilidades originais e classifica cada item como `confirmed`, `confirmed_static`, `not_confirmed` ou `inconclusive`.

Etapa 5 - Diagnóstico e Correção: Para vulnerabilidades confirmadas, a LLM gera `DiagnosticoFalhas` e depois um Solidity completo corrigido.

Etapa 6 - Revalidação: O contrato corrigido é verificado pelo Certora; o resultado é comparado com as vulnerabilidades confirmadas. O loop roda até `MAX_TENTATIVAS_CORRECAO = 3`.

Publicação do Fix: Se não houver falhas persistentes ou inconclusivas, o arquivo temporário em `agent_outputs` é copiado para `smart-audit/contracts/_FIXED.sol`.

4. Tecnologias e Versões

Tecnologia	Versão detectada	Uso no pipeline
Python	Python 3.10.12	Runtime do agente.
Slither	0.11.3	Análise estática de contratos Solidity.
Certora CLI	certora-cli 8.1.1	Execução do Certora Prover.
solc	Version: 0.8.21+commit.d9974bed.Linux.g++	Compilador Solidity usado pelo fluxo local.
OpenAI SDK	2.36.0	Cliente Python usado para chamar Groq via API compatível com OpenAI.
Pydantic	2.11.7	Validação das respostas estruturadas da LLM.
ReportLab	3.6.8	Geração deste PDF baseline.

5. LLM e Provedor Atual

O código atual não usa Ollama/local LLM. Ele usa Groq hospedado por meio do OpenAI SDK:

```
base_url = https://api.groq.com/openai/v1
model = llama-3.3-70b-versatile
env = GROQ_API_KEY
```

A consequência prática é que o modelo é da família Llama, mas a inferência depende do serviço Groq. Uma melhoria futura recomendada é abstrair provedores com `LLM_PROVIDER=groq|ollama|openai-compatible`, preservando Groq como baseline forte e permitindo experimentos locais.

6. Artefatos Existentes

Contratos em smart-audit/contracts:

- `AnotherVulnerableBank.sol`
- `CrowdfundingVault.sol`
- `DeFiVault.sol`
- `DeFiVault_FIXED.sol`
- `King.sol`
- `SafeBankToken.sol`

- SafeBankToken2.sol
- SimpleBank.sol
- SimpleBank_FIXED.sol
- SuperSecureBank.sol
- TimelockVault.sol
- TreasuryVault.sol
- VulnerableBankToken.sol
- VulnerableBankTokenCorrected.sol
- VulnerableVault.sol

Specs CVL em specs:

- DeFiVault.spec
- VulnerableBankToken.spec
- VulnerableBankTokenCorrected.spec
- VulnerableVault.spec
- another_vulnerable_bank.spec
- banktoken.spec
- banktoken_reentrancy.spec
- crowd_funding_vault.spec
- reentrancy_corrected.spec
- timelockVault.spec
- treasuryvault.spec

Arquivos Python principais do agente:

- agent/llm/client.py
- agent/llm/schemas.py
- agent/main.py
- agent/orchestrator.py
- agent/prompts/system_prompts.py
- agent/tools/certora_cli.py
- agent/tools/slither_cli.py
- agent/tools/spec_validator.py

Artefatos gerados observados em agent/agent_outputs:

- DeFiVault.spec
- DeFiVault_FIXED.sol
- certora_fix_log_t1.txt
- certora_original_log.txt
- comparacao_t1.json
- diagnostico_t0.json
- etapa1_vulns.json
- etapa4_analise.json
- etapa6_analise_t1.json

7. Divergencias e Problemas Conhecidos

- README e código divergem em um ponto central: o README descreve spec CVL validada/manual, mas o orchestrator atual gera spec via LLM e autocorrigge depois.

- O pipeline pode classificar resultados com base em logs mesmo quando o Certora falhou por erro de spec ou compilacao; isso pode gerar conclusoes enganosas.
- validar_spec emite avisos, mas o orchestrator continua mesmo quando valido=False.
- A analise de logs do Certora depende da LLM, embora parte do status possa ser extraida deterministicamente.
- O mapeamento de vulnerabilidades ainda e pequeno e acoplado aos nomes do Slither.
- Casos semanticos como ERC2771 + multicall + delegatecall podem aparecer como low-level-calls/calls-loop, mas nao como uma vulnerabilidade especifica.
- A correcao de Solidity e gerada como texto livre; isso aumenta risco de mudanca de logica, comentarios obsoletos e correcoes no alvo errado.
- Comentarios VULN em contratos corrigidos podem confundir etapas posteriores se o texto bruto do contrato for usado como evidencia.
- agent_outputs, env-* e logs devem continuar fora do git ou ir para uma area de experimentos controlada.

8. Melhorias Recomendadas

- Separar claramente manual, automatizado, exemplos e artefatos gerados.
- Adicionar uma camada de configuracao para escolher contrato, spec, modelo, provedor LLM e diretorio de saida sem editar orchestrator.py.
- Criar um parser deterministico para logs do Certora antes de pedir interpretacao a LLM.
- Fazer o pipeline parar quando Certora retorna erro de spec/compilacao, antes de diagnosticar vulnerabilidade.
- Trocar prompts monoliticos por templates menores: normalizacao, spec, diagnostico e patch com contratos de saida mais restritos.
- Adicionar catalogo de vulnerabilidades com fontes publicas e templates CVL por tipo.
- Suportar Groq e Ollama/local via uma interface comum de provedor LLM.
- Criar testes pequenos para spec_validator, slither_cli e parser de Certora.
- Manter develop como branch de trabalho e main como baseline estavel.

9. Estado Git Depois da Preparacao

- Branch ativa: develop.
- main, develop e HEAD apontam para fda0435.
- Mudancas locais rastreadas e untracked comuns foram preservadas em stash.
- O diretorio smart-audit/aave-v4 e um repositorio Git aninhado; o Git do repo pai o ignorou no stash. Ele foi mantido no filesystem e estava limpo internamente no momento da verificacao.
- Este PDF e o primeiro artefato novo gerado em develop apos o alinhamento.

10. Leitura Executiva

O pipeline ja possui a espinha dorsal certa para a pesquisa: Slither encontra sinais, a LLM estrutura e repara, e Certora tenta provar se a correcao eliminou a falha. O principal risco atual nao e a ideia, mas a confiabilidade da automacao ao redor da prova formal. A proxima etapa deve reduzir ambiguidade: logs parseados por codigo, falhas formais tratadas como bloqueantes, catalogo de vulnerabilidades, specs por template e provider LLM configuravel.